ArchFlow Dashboard Operating Manual

Status: local/static dashboard operating document

Date: 2026-07-02

Purpose

The dashboard is the local control and communication surface for ArchFlow's PRD automation service and internal agent workflow. It has two architectures:

- Architecture 1 - PRD/ICP Service Output: turns approved source material into PRD, task matrix, evidence gaps, ICP logic, reports, and review packets.
- Architecture 2 - Agent Control System: controls Codex/local execution, LangGraph-style routing, LlamaIndex retrieval, Graphify structure reference, WikiLLM memory candidates, reviews, approvals, run notes, and deployment sequencing.

The dashboard is still static/browser-local by default. It can create local packets, persist browser-local chat, and call the local provider-disabled Jarvis API when that API is running. It does not safely perform provider calls, Railway deployment, Notion writeback, Telegram sends, raw voice storage, or production publication without separate approval and runtime proof.

First Screen: Jarvis

Architecture Selector

Use the Architecture selector before giving commands.

Option	Use For	Effect
Architecture 1	PRD/ICP service work	Jarvis treats commands as source-to-PRD, discovery synthesis, ICP evidence, backlog, and buyer-output requests.
Architecture 2	System/control work	Jarvis treats commands as architecture, workflow, logs, memory, Graphify, LlamaIndex, LangGraph, approval, and durable-output requests.

Normal Mode

Normal mode waits for a command. The same sentence has different meaning depending on the selected architecture. In Architecture 1, "run a check" creates a PRD/ICP service packet. In Architecture 2, it creates an architecture-control packet with logs, approval gates, and memory/writeback boundaries.

Interview Mode

Interview mode is proactive. When selected, Jarvis immediately asks the first question for the active architecture.

Architecture 1 starts by asking which source material should become the next PRD/ICP packet and what buyer/product decision it should support.

Architecture 2 starts by asking what system-level change, agent workflow, memory/logging decision, or dashboard-control outcome should be structured first.

Each answer is stored as a browser-local summary candidate and the next question is shown. Raw transcript persistence remains off by default.

Chat History

Chat history is persistent in local browser storage. It stays available across page reloads until the user explicitly clears it. Export chat history before it should become a Codex-reviewed packet.

Clear history removes only the browser-local copy. It does not delete Git, Notion, WikiLLM, or run artifacts.

Voice And Transcript Controls

Voice input uses the browser speech recognition API when available. Manual transcript fallback uses the same Jarvis chat path. Voice output uses browser speech synthesis when enabled.

Current limits:

- Raw audio is not stored.
- Raw transcript persistence is off by default.
- Provider-backed transcription or TTS is not enabled.
- Owner approval and storage policy are required before durable voice capture.

Screen 1: PRD/ICP Flow

Screen 1 is the buyer-output flow. It shows how source material becomes a PRD/ICP service artifact.

Main stages:

1. Client source intake.
2. PRD builder.
3. Market evidence fork.
4. ICP evidence cards.
5. Customer pain review.
6. Evidence merge.
7. ICP synthesis.
8. Landing/demo package.
9. Runtime proof branch.
10. Client-output approval.
11. Service output packet.

Each stage should define:

- Inputs.
- Outputs.
- Owner.
- Evidence or pending marker.
- Business objective.
- Review gate.
- Runtime boundary.

Screen 2: Agent Orchestra

Screen 2 is the internal control workflow. It is for local agent coordination, not buyer-facing output.

Main stages:

1. Operator command intake.
2. Classify request.
3. Codex development response.
4. Graph monitor.
5. Parallel agent fork.
6. Architecture review.
7. Safety and source review.
8. CrewAI Level 3 direct proof.
9. Integrator merge.
10. Writeback approval.
11. Durable output.

Block-Schema Nodes

Agent Nodes

Agent nodes represent bounded work owned by a role. Every agent node should have inputs, outputs, files, prompt, system prompt, last run state, possible outputs, and safety requirements.

Router Nodes

Router nodes classify work before execution. They decide whether a command is PRD/ICP service work, architecture/control work, safety review, provider-gated runtime work, or blocked work.

Parallel Nodes

Parallel nodes are not decoration. They mean that the integrator may split work into independent branches, such as architecture review, safety review, product review, and technical runtime review. A parallel node must define branch owners, inputs, outputs, merge target, and stop conditions.

Approval Nodes

Approval nodes protect high-impact actions. They are required before provider calls, raw capture, external sends, Notion/GitHub/WikiLLM writeback, Railway deployment, production promotion, or public claims. Approval outputs should be approved action, revision request, or blocked issue.

Merge Nodes

Merge nodes combine branch outputs and preserve contradictions as gaps. The merge node does not hide blockers; it makes one accepted handoff after review.

Output Nodes

Output nodes create durable artifacts: run notes, PRDs, reports, PDFs, issues, decisions, dashboard data refreshes, or Git commits.

Config Page

The Config page stores browser-local prompt and role settings. These are candidates only. They do not change GitHub, Notion, WikiLLM, provider prompts, or deployed services until Codex reviews and writes the change.

Important config fields:

Field	Meaning	Risk
Model policy	Defines allowed provider state	Must stay provider-disabled unless approved
Memory policy	Defines what can be stored	Must exclude raw private source by default
Normal prompt	Reactive command behavior	Should respect selected architecture
Interview prompt	Proactive question behavior	Should ask one question at a time
Review prompt	Safety and evidence gate	Must block unsupported claims

Local Jarvis API

The local API lives under `services/jarvis-api`. It is provider-disabled by default.

Expected local checks:

- `/health`
- `/api/config/roles`
- `/api/lanes/prd-icp`
- `/api/lanes/agent-orchestra`
- `/api/test-runs/meeting-prd`
- `/api/voice/chat`

The API can return review packets and approval gates. It does not call OpenRouter, write Notion/WikiLLM/GitHub, store raw transcripts, or run full provider-backed PRD cycles without approval.

Railway Readiness

Railway is the future hosted runtime lane. It is not complete until all of the following are proven:

- Only `services/jarvis-api` is deployed.
- Hosted `/health` is verified.
- CORS and auth are configured.
- Provider calls remain disabled first.
- Dashboard API routing points to the hosted backend only after review.
- Logs and budget guards exist.
- Owner approval is recorded before provider activation.

Required Operator Checks

Before Git push or Notion status promotion:

- Public safety scan.

- Workflow validation.
- Runtime guard.
- Dashboard JSON parse.
- JavaScript syntax check.
- API syntax check.
- Dashboard browser smoke test at desktop and mobile widths.
- PDF existence and size check for generated report documents.

Status Rules

- Done: bounded output exists, checks pass, and no owner approval remains.
- Review: artifacts exist but owner acceptance, hosted proof, or final closeout is pending.
- Blocked: required input, approval, provider gate, or proof is missing.
- Backlog: planned but not approved or started.

Current Boundaries

The dashboard can guide and stage work locally. Codex remains the operator/editor/reviewer. LangGraph, CrewAI, LlamaIndex, Graphify, WikiLLM, OpenRouter, Railway, Notion, Telegram, and Figma each require their own verified path before claims are promoted.